24-Day Professional Schedule: Multi-Tenant Workflow Automation Project

Day 1 (Mar 6): Implement Authentication Service and Login Endpoint

- **Objective:** Set up user authentication so users can log in and receive a JWT. Ensure only authenticated requests can access protected APIs.
- **File Locations/Setup:** Create AuthService.java in src/main/java/.../service and AuthController.java in .../controller. Use the existing UserRepository. Configure the login endpoint (e.g. /api/auth/login) in Spring Security to be public.
- **Core Concepts:** Use JSON Web Tokens (JWT) for stateless authentication. JWTs securely encode user identity claims so backend can verify requests without server-side sessions. A generated JWT is returned on successful login and included in future requests' headers.
- **Implementation Steps:**
    1. In AuthService, implement a login method that checks the provided username/password via UserRepository. Use a password encoder (e.g. BCrypt) and compare with stored hash.
    2. If valid, call JwtService.generateToken(user) to create a JWT. Return the token.
    3. In AuthController, add a POST /login endpoint that accepts credentials, calls AuthService.login, and returns the JWT (typically in JSON).
    4. Update SecurityConfig to permit the /login path and to use your JwtAuthenticationFilter for other endpoints.
- **If Skipped:** Without an AuthService or login API, clients cannot obtain a JWT, and all requests to protected endpoints will be unauthorized. The backend remains locked down with no way to authenticate users.

Day 2 (Mar 7): Verify Authentication Flow and User Setup

- **Objective:** Test and refine the authentication flow. Create at least one user for login and ensure the JWT and filter work correctly.
- **File Locations/Setup:** Add a CommandLineRunner or SQL migration to insert a default test user into the database. Use application.properties to configure datasource (already done).
- **Core Concepts:** Confirm password hashing and error handling. The global error handler can manage bad-credentials responses gracefully.
- **Implementation Steps:**
    1. Ensure a user (with encrypted password) exists. For example, write a data.sql or use a CommandLineRunner bean to insert a user.

2. Use a tool like Postman to POST to /api/auth/login with credentials. Verify the response contains a valid JWT.
3. Use the JWT to call a protected endpoint (e.g. /health). Ensure it succeeds when the token is valid, and fails when invalid or absent.
4. Handle login failures: if credentials are wrong, throw a custom exception (handled by your global error handler) to return a 401.

- **If Skipped:** If the authentication flow isn't verified, the system may have broken login logic or no test user. This would block all further testing and usage of the application.

Day 3 (Mar 8): Define Workflow Entity and Enable Multi-Tenancy

- **Objective:** Model the Workflow resource in the database and enforce tenant isolation. Each workflow belongs to a tenant (user).
- **File Locations/Setup:** Create Workflow.java in .../model (with fields like id, name, definition, tenantId) and WorkflowRepository.java in .../repository. Ensure JPA sees the new entity.
- **Core Concepts:** Use PostgreSQL Row-Level Security (RLS) to isolate tenants. With RLS enabled, policies can ensure a tenant only sees its own data. Include a tenant_id column in the workflow table that RLS will filter on. The application should set the current tenant (e.g. via SET ROLE) when a user logs in, or include tenant_id in queries.
- **Implementation Steps:**
    1. Define the Workflow JPA entity with a tenantId field (mapped to tenant_id column). Annotate with @Entity and include other necessary fields.
    2. Enable RLS in the database for the workflow table (using SQL: ALTER TABLE workflow ENABLE ROW LEVEL SECURITY; CREATE POLICY tenant_isolation ON workflow USING (tenant_id = current_setting('myapp.current_tenant')::integer);).
    3. In the application, after authentication, set the session variable for the tenant (e.g. call a SQL function or use a Hibernate interceptor to run SET myapp.current_tenant = <userTenantId>). This ensures RLS filters queries automatically.
    4. Create WorkflowRepository as a JPA repository. No special query logic is needed because RLS enforces filtering.
- **If Skipped:** Without modeling Workflow, there is nowhere to store workflows. Without tenant isolation (RLS), users could see or modify other tenants' workflows, breaking the multi-tenant requirement.

Day 4 (Mar 9): Implement Workflow CRUD Endpoints

- **Objective:** Provide REST endpoints to Create, Read, Update, and Delete workflows. Ensure these respect tenant boundaries.
- **File Locations/Setup:** Create WorkflowController.java in .../controller. If needed, add a WorkflowService.java in .../service to encapsulate business logic. Use WorkflowRepository for DB operations.
- **Core Concepts:** RESTful API design with Spring MVC. Each CRUD operation must automatically filter by the current tenant (handled by RLS). Typical routes:
    - POST /api/workflows to create
    - GET /api/workflows to list tenant's workflows
    - GET /api/workflows/{id} to view one
    - PUT /api/workflows/{id} to update
    - DELETE /api/workflows/{id} to delete.
- **Implementation Steps:**

1. In WorkflowController, add methods for each route. For example, @PostMapping("/workflows") to create a new Workflow from request body, save via repository.save().

2. When returning lists or single items, let RLS do the filtering (no extra code needed for tenant checking).

3. Handle cases where a workflow is not found or not owned by tenant (RLS will make it not found). Return appropriate HTTP status (404 or 403) for unauthorized access.

4. Optionally, validate input (e.g. non-empty name).

- **If Skipped:** If no CRUD API, users cannot define or manage workflows. The execution engine then has nothing to operate on. The system's functionality would be incomplete.

Day 5 (Mar 10): Design Workflow Execution Model

- **Objective:** Plan and partially implement how a workflow is executed. Introduce entities to track executions and steps.
- **File Locations/Setup:** Create WorkflowExecution.java in .../model (fields: id, workflowId, status, startTime, endTime, tenantId). Optionally create TaskExecution.java to log individual step results.
- **Core Concepts:** A workflow is a set of tasks with dependencies (a DAG). We need an execution engine that can run these tasks, respecting order. Using a DAG execution

engine (ExecutorService) ensures tasks run in correct order and can run in parallel where possible. We also persist execution status.

- **Implementation Steps:**
    1. Define WorkflowExecution entity with a many-to-one relationship to Workflow. Include status (e.g. PENDING, RUNNING, SUCCESS, FAILED) and timestamps. Link it to the same tenantId for isolation.
    2. Create a repository WorkflowExecutionRepository.
    3. Sketch a service WorkflowExecutionService or WorkflowExecutor class. This will fetch a workflow's definition and drive its execution. For now, focus on data model; actual execution logic comes next.
    4. Consider how to represent workflow steps. If workflows are defined in JSON or DB, parse them into in-memory nodes. If steps are in another table, set that up.
- **If Skipped:** Without an execution model, the system cannot run workflows or track progress. There would be no records of runs or any way to report success/failure.

Day 6 (Mar 11): Implement Basic Sequential Execution Engine

- **Objective:** Code the first version of the workflow engine: execute tasks one-by-one in defined order.
- **File Locations/Setup:** In WorkflowExecutor.java (or within WorkflowExecutionService), implement a method like executeWorkflow(Long workflowId). Use existing service classes for tasks (AI service, etc.).
- **Core Concepts:** Initially execute tasks sequentially. For each step in the workflow definition, perform the action synchronously before moving to the next. This simple approach establishes a working engine.
- **Implementation Steps:**
    1. In the execute method, load the Workflow by ID and record a new WorkflowExecution (status RUNNING).
    2. Parse the workflow's steps (this might involve deserializing a JSON definition or similar).
    3. Loop through steps in order. For each step, call the corresponding handler (e.g. if it's an "AI" step, call aiAdapter.callAi(...); if it's a "task" step, do the task logic).
    4. After each step, optionally update TaskExecution log and continue. If any step throws an exception, set workflow status to FAILED and break.

5. If all steps finish, mark the WorkflowExecution as SUCCESS and set end time.

- **If Skipped:** Without any execution logic, workflows would never run. The core feature (automation) would be missing, and the project would just be a data storage with no action.

Day 7 (Mar 12): Enhance Execution with Concurrency (DAG Engine)

- **Objective:** Upgrade the engine to run independent tasks in parallel using a DAG-based approach.
- **File Locations/Setup:** Update WorkflowExecutor.java to manage concurrency. Possibly add helper classes like WorkflowNode (as in a DAG) or use an adjacency list.
- **Core Concepts:** Convert the workflow into a Directed Acyclic Graph (DAG). Tasks with no dependencies can run immediately, independent tasks run concurrently via ExecutorService. This maximizes efficiency and still respects order.
- **Implementation Steps:**
    1. Represent each workflow step as a WorkflowNode with a list of dependency IDs. Build the DAG by adding nodes and edges.
    2. Initialize an ExecutorService (e.g. fixed thread pool).
    3. Identify all nodes with no unmet dependencies (in-degree = 0) and submit them to the executor. Keep a map of Futures.
    4. When a task completes, remove it from the in-progress list and decrement the in-degree of dependent nodes. If any dependent node's in-degree drops to zero, submit it to the executor.
    5. Continue until all tasks complete or fail. Collect results and update WorkflowExecution.
- **If Skipped:** The engine would remain strictly sequential. That limits performance (no parallelism) and doesn't demonstrate full automation capabilities. For complex workflows, this would be inefficient.

Day 8 (Mar 13): Add Retry and Failure Handling Logic

- **Objective:** Make the engine robust by implementing retry policies and fault tolerance for transient failures.
- **File Locations/Setup:** In WorkflowExecutor.java, wrap task submissions with retry logic. Possibly add configuration in application.properties for retry limits.
- **Core Concepts:** Workflows should be resilient. Retries help recover from temporary errors, but only if tasks are idempotent. Ensure that retrying a task won't cause

duplicate side effects. Track an operation ID for each step so that retries do not repeat work (using a unique key per task execution).

- **Implementation Steps:**
    1. For each task execution, catch exceptions. If a task fails due to a recoverable error, retry it up to N times (e.g. 3). Implement backoff (e.g. exponential sleep).
    2. Use an idempotency key (e.g. combination of workflowExecutionId and stepId) to check if this step was already completed in a previous attempt. If so, skip re-executing the same operation.
    3. If a task ultimately fails after retries, mark the WorkflowExecution as FAILED and stop further tasks. Log the error.
    4. Ensure that retry parameters (limits, backoff) are configurable.
- **If Skipped:** Without retries, a transient glitch (e.g. network blip, temporary service outage) would cause the entire workflow to fail. Tasks that are not idempotent might accidentally execute twice on retry, causing incorrect results.

Day 9 (Mar 14): Integrate AI Module into Workflow Steps

- **Objective:** Enable workflow steps that call AI services. Incorporate the basic AI module scaffolding into the execution engine.
- **File Locations/Setup:** Implement AIAdapter.java or similar in .../service to interface with the AI library or API. Update WorkflowExecutor to detect AI-type steps.
- **Core Concepts:** AI tasks can be treated like any other task node. When a workflow step is of type "AI", call the AI adapter which fulfills a pre-defined contract (e.g. takes a prompt and returns text). The AI call should be asynchronous or blocking as fits.
- **Implementation Steps:**
    1. In the workflow definition, mark tasks that require AI processing (e.g. a field type: AI with parameters).
    2. In WorkflowExecutor, when dispatching nodes, if a node is AI-type, call the aiAdapter.process(inputData) method. The AIAdapter uses the already-scaffolded AI module to get a result.
    3. Handle the AI response by storing it (e.g. in TaskExecution) or passing it to subsequent steps.
    4. Ensure any exceptions from the AI call are caught and trigger the same retry/failure logic.

- **If Skipped:** Any workflow tasks requiring AI (e.g. text generation, analysis) would not work. The feature "AI contract integration" would be missing, making those workflows effectively no-ops.

Day 10 (Mar 15): End-to-End Backend Testing of Workflow Execution

- **Objective:** Test the full execution pipeline, especially AI-related tasks, and fix any issues. Ensure multi-tenant context throughout.
- **File Locations/Setup:** Use src/test/java/... for writing integration tests (e.g. with Spring's @SpringBootTest) or manual testing via Postman.
- **Core Concepts:** Validate that a defined workflow actually completes. For multi-tenancy, confirm that each execution is isolated by tenant. Use logs and the database to trace execution steps.
- **Implementation Steps:**
    1. Create a test workflow (via the CRUD API) that includes an AI step and maybe a simple system step.
    2. POST to /api/workflows/{id}/execute (assuming you create such an endpoint) or call the execute service method.
    3. Observe the returned WorkflowExecution status after some time (poll or asynchronous). Check the database: a new execution record should be SUCCESS, and any TaskExecution records show completed steps.
    4. If any step failed, use logs to diagnose. Fix code (e.g. adjust task parsing, AI call exceptions). Ensure that after fixes the workflow completes.
    5. Also test a failing scenario (e.g. invalid step) to see that error handling works.
- **If Skipped:** Unverified code may have critical bugs. Without testing, there's no guarantee the engine actually runs workflows correctly or that AI integration functions as intended.

Day 11 (Mar 16): Logging, Monitoring, and Error Refinement

- **Objective:** Improve observability. Add detailed logging and refine the global error handler so issues are easy to diagnose during demo.
- **File Locations/Setup:** Configure logback.xml or use logger in classes. Possibly update GlobalExceptionHandler.java.
- **Core Concepts:** Good logs and errors help demonstrate and debug the system. Each major action (start/finish execution, task success/failure) should log with context (workflowId, tenantId) so the run can be followed in console or files.
- **Implementation Steps:**

    1. Insert logging statements (e.g. using SLF4J) at the start and end of executeWorkflow, and in each task execution. Include identifiers.
    2. In exception handlers, log stack traces. Customize error messages to include the step that failed.
    3. Update GlobalExceptionHandler to catch any unhandled exceptions during execution and convert to a user-friendly response.
    4. Optionally, emit simple metrics or counts (logs of execution start/end suffice for now).
- **If Skipped:** Without good logs, it will be hard to explain or debug what happens during a demo. Failures may appear silent or confusing, reducing confidence in the solution.

Day 12 (Mar 17): Frontend – React Login Page

- **Objective:** Begin the minimal React frontend by implementing a login screen that obtains a JWT from the backend.
- **File Locations/Setup:** In the React app (e.g. frontend/src/), create components/Login.js. Set up routing (use react-router-dom) with a route for /login. Ensure Axios (or fetch) is configured to call the backend.
- **Core Concepts:** Store the JWT in browser storage (localStorage or sessionStorage) so it persists across refreshes. This implements a token-based auth flow on the client. Use a simple form to collect credentials.
- **Implementation Steps:**
    1. Scaffold a React project (using Create React App) if not done. Install React Router v6.
    2. Build a Login component with username/password fields and a submit button.
    3. On submit, POST the credentials to the backend /api/auth/login using Axios.
    4. On successful response, save the JWT in localStorage (e.g. localStorage.setItem("token", jwt)). Then redirect to a protected route (like /workflows).
    5. Handle errors (invalid credentials) by showing a message on the form.
- **If Skipped:** Without a login UI, there is no way for demo users to authenticate via the frontend. The front end cannot call protected APIs, and the demo would lack a web interface for the core flow.

Day 13 (Mar 18): Frontend – Workflows List and Protected Routes

- **Objective:** Display the authenticated user's workflows. Protect this route so only logged-in users can access it.

- **File Locations/Setup:** Create components/Workflows.js. Update App.js (or routing component) to include a route for /workflows. Wrap routes in an "AuthRoute" or use a context/hook.
- **Core Concepts:** Use React Router v6 to enforce protected routes. On route load, read the JWT from localStorage and attach it as a Bearer token in API calls. A common pattern is a wrapper or context that checks localStorage before rendering the protected component.
- **Implementation Steps:**
    1. Create a Workflows component that fetches /api/workflows on mount (useEffect). Include the JWT in the Authorization header (Authorization: Bearer <token>).
    2. In App.js, define a <Routes> with a route for /workflows. Protect it by checking if token exists; otherwise redirect to /login. (This follows the "Protected routes" pattern.)
    3. In Workflows, render a table or list of workflows (ID, name, etc.) returned by the API.
    4. Add a "Create Workflow" button or form if desired (optional, since focus is backend). At minimum, show existing workflows.
- **If Skipped:** The front end would not display any workflows or enforce login. The demo would lack a UI for showing and selecting workflows to run, and users might access data they shouldn't.

Day 14 (Mar 19): Frontend – Trigger Workflow Execution

- **Objective:** Allow the user to trigger a workflow run from the UI and observe basic feedback (e.g. status message).
- **File Locations/Setup:** Update Workflows.js or add a new Execution.js. Use the existing workflows list.
- **Core Concepts:** Send a request to start execution. For example, a POST to /api/workflows/{id}/execute (create this endpoint in backend if not already). After starting, poll or redirect to view execution status.
- **Implementation Steps:**
    1. In the workflows list, add a "Run" button next to each workflow entry. When clicked, call the backend execute endpoint for that workflow ID using the stored JWT.
    2. Upon success, display a message (e.g. "Execution started"). Optionally, redirect to an execution details page.

3. If implementing, add a route /executions/:id or reuse /workflows/:id to show execution history (next step).
4. Ensure UI handles failures (e.g. show error if execution could not start).

- **If Skipped:** There would be no way to run workflows through the UI; the demo could only show CRUD operations. The key automation feature (running workflows) would not be demonstrable in the front end.

Day 15 (Mar 20): Frontend – Show Execution History/Results

- **Objective:** Display past executions of a workflow and their status/results.
- **File Locations/Setup:** Create components/Executions.js (or expand Workflows.js). Add a route /executions or nested routes like /workflows/:id/executions.
- **Core Concepts:** Fetch execution data from the backend (e.g. GET /api/workflows/{id}/executions) and show it. This lets users see the outcome of runs.
- **Implementation Steps:**
  1. In the backend, ensure there is an endpoint to list executions for a workflow (e.g. GET /api/workflows/{id}/executions). If not, add it in WorkflowController.
  2. In React, after running a workflow or clicking a workflow, navigate to a page that loads this data. For example, on click of a workflow name, go to /workflows/:id which shows a list of executions.
  3. Fetch the executions list with JWT in header. Display each execution ID, start time, end time, status (e.g. SUCCESS/FAILURE).
  4. Optionally, allow clicking an execution to see detailed logs or results (advanced). For now, a list suffices.
- **If Skipped:** The demo would not show that the workflow actually ran. It would lack closure, leaving the audience unsure if execution worked. Displaying results is important for completeness.

Day 16 (Mar 21): Backend Testing and Validation

- **Objective:** Write unit and integration tests for the backend services and controllers to ensure correctness.
- **File Locations/Setup:** Use JUnit (with Spring Boot Test) in src/test/java/.... Add test classes like AuthServiceTest, WorkflowControllerTest, etc.
- **Core Concepts:** Automated testing ensures reliability. Cover core paths: successful and failed login, CRUD operations (including RLS filtering), and a sample execution run. Mock the database or use an in-memory DB for tests.
- **Implementation Steps:**

    1. Test AuthService: given valid credentials, assert a token is returned. Given invalid credentials, assert exception thrown.
    2. Test WorkflowController: with a mock user/tenant, test that creating a workflow persists and listing returns only that tenant's workflows.
    3. If possible, write an integration test for the execution engine: create a simple workflow, call the execute method, and verify a WorkflowExecution record is created with correct status.
    4. Test RLS: simulate two tenants and show that tenant A cannot see tenant B's workflows. This may involve setting up two data sources or roles in tests.
- **If Skipped:** Bugs or regressions are likely. Without tests, any change could break functionality undetected, risking a failed demo or late issue.

Day 17 (Mar 22): Refactoring and Performance Tuning

- **Objective:** Polish code: refactor for readability, improve performance (e.g. thread pool sizing), and ensure thread-safety.
- **File Locations/Setup:** Review all service classes. Optionally configure thread pool size in application.properties.
- **Core Concepts:** As a backend intern, focus on clean code and efficiency. For instance, using a fixed thread pool of size matching expected concurrency. Ensure singleton services are thread-safe when called by multiple threads.
- **Implementation Steps:**
    1. Walk through the code, refactor any duplicate logic into methods. Clean up large methods for maintainability.
    2. Adjust the ExecutorService pool size if needed based on number of cores. (4-10 is reasonable for 4 CPU).
    3. Ensure that any shared data structures in WorkflowExecutor are properly synchronized or confined to a single thread.
    4. Review database access: batch updates if any, and add indexes if queries are slow (e.g. on tenant_id).
- **If Skipped:** The solution might work functionally but be messy or inefficient. In a real project, this would lead to maintenance headaches or slow performance under load.

Day 18 (Mar 23): Dockerize the Application

- **Objective:** Containerize the backend (and database) for easy deployment.

- **File Locations/Setup:** Create a Dockerfile in the project root and a docker-compose.yml. The Dockerfile should use a multi-stage build: first build the JAR, then use a slim Java base image.
- **Core Concepts:** Docker images ensure consistency. Use an official OpenJDK base image. Mount volumes or use docker-compose for Postgres with RLS configuration. Expose the app port (e.g. 8080).
- **Implementation Steps:**
    1. Write Dockerfile:

```
FROM maven:3.8-openjdk-21 as builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn package -DskipTests
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=builder /app/target/myapp.jar .
ENTRYPOINT ["java","-jar","myapp.jar"]
```

    2. Write docker-compose.yml with services for app and db. For Postgres, include environment (POSTGRES_DB, user, password) and volumes.
    3. In docker-compose, ensure initialization SQL scripts (with RLS policies) are run on DB startup (using docker-entrypoint-initdb.d/).
    4. Test building the image: docker-compose build.
- **If Skipped:** The app cannot easily be run in a controlled environment. Dependency on local setup means demo might fail due to “it works on my machine” issues.

Day 19 (Mar 24): Docker Compose Testing

- **Objective:** Run the entire application stack via Docker to verify it works end-to-end.
- **File Locations/Setup:** Use the docker-compose.yml created. Optionally add a .env file for configs.
- **Core Concepts:** A running container environment should mirror production. Check that RLS policies are in effect (Postgres recognizes the tenant_id setting).
- **Implementation Steps:**
    1. Run docker-compose up and wait for services to initialize.
    2. Once the app container is up, access it (e.g. http://localhost:8080/health) to verify connectivity.

3. Use the React app (served by a dev server or container) or Postman to hit the APIs as if remote. Test login, workflow CRUD, execution, etc., now through the container network.
4. If any issues (e.g. DB not initialized with RLS scripts), fix the Docker setup (maybe adjust init scripts or network settings).

- **If Skipped:** Deployment issues (environment mismatch, missing scripts) may go unnoticed until the demo. Running locally is different from running in Docker.

Day 20 (Mar 25): Demo Preparation and Script

- **Objective:** Prepare a clear demo script and final UI touches. Ensure everything is ready to show.
- **File Locations/Setup:** No code changes required; prepare a document outlining demo steps (login, create workflow, execute, show results). Polish front-end labels if needed.
- **Core Concepts:** A good demo is organized and highlights features. Plan a narrative: e.g., "First, we log in. Then we create a workflow. Next we run it and watch the results."
- **Implementation Steps:**
    1. Write down a step-by-step demo checklist. For example:
        - Login as Tenant A user.
        - Create a new workflow via UI.
        - Show that workflow in list.
        - Run the workflow and show execution status.
        - Show the log or execution history.
        - Optionally, login as Tenant B to show RLS isolation.
    2. Practice the demo at least once to ensure timing and that every feature works as expected.
    3. Make final minor tweaks (UI labels, documentation comments, README).
- **If Skipped:** The demo may feel disorganized or some features may be missed. Lack of preparation often leads to mistakes or forgetting key points during the live presentation.

Day 21 (Mar 26): Final Bug Fixes and Polish

- **Objective:** Use remaining time to fix any bugs found during demo practice and add small improvements.

- **File Locations/Setup:** As needed—likely code touched in previous days.
- **Core Concepts:** Improve user experience and stability. For example, catch any last exceptions, style the login form, or add loading indicators.
- **Implementation Steps:**
    1. Address any errors discovered during testing (e.g. null pointers, UI glitches).
    2. Add edge-case checks (e.g. disallow empty workflow names, disable Run button while running).
    3. Optionally, add simple CSS or layout fixes to make the UI cleaner.
- **If Skipped:** Unfixed bugs or rough UI issues could detract from the demo's professionalism and cause unnecessary confusion.

Day 22 (Mar 27): Documentation and README

- **Objective:** Write a README and any documentation so others can understand and run the project.
- **File Locations/Setup:** Create or update README.md at project root. Include instructions for setup, running (including Docker), and using the APIs.
- **Core Concepts:** Clear documentation is key for maintainability. Mention tech stack, how to run (Maven vs Docker), and important endpoints.
- **Implementation Steps:**
    1. In README.md, write sections: Project Overview, Prerequisites, Setup & Run (including docker-compose up), API Endpoints (list the main ones like /login, /workflows, /execute), and Frontend Usage.
    2. Include notes on multi-tenancy (e.g. "Every request is tenant-scoped via RLS").
    3. Optionally, provide curl examples or screenshots of the UI.
- **If Skipped:** New developers or evaluators may be unable to run or understand the project without guidance. Poor documentation can make a project look incomplete or amateurish.

Day 23 (Mar 28): Final End-to-End Test

- **Objective:** Perform a complete run-through from fresh environment to ensure everything still works.
- **File Locations/Setup:** As before (could even tear down and rebuild Docker).
- **Core Concepts:** Verify idempotency by running the same steps twice; ensure no leftover state. Check logs for unexpected errors.

- **Implementation Steps:**
    1. Shut down and restart Docker to simulate a fresh start.
    2. Go through the demo script from login to execution, as if seeing it for the first time.
    3. Test any edge cases (e.g. login with wrong password, running a workflow twice).
    4. Confirm that nothing crashes or misbehaves. Fix any last-minute issues.
- **If Skipped:** There's a risk of discovering a critical issue only during the actual demo. This final test is the last chance to avoid such surprises.

Day 24 (Mar 29): Rest and Final Preparations

- **Objective:** Light day for rest or final touches. Ensure slides or talking points are ready if needed. Backup all code.
- **Core Concepts:** It's better to be slightly over-prepared. Have the database reset scripts ready, and an "undo" plan if something in demo goes wrong.
- **Implementation Steps:**
    1. Backup the code to a remote repo (GitHub/GitLab) if not already done.
    2. Prepare any presentation slides or diagrams if the demo requires explanation.
    3. Double-check environment variables and configurations.
- **If Skipped:** You might rush into the demo day unprepared or without a fallback plan. Taking a step back ensures you go in calm and confident.